BRAC University

CSE221: Algorithms | Assignment (1)

---

1. a. Analyse the following code segment and **express** the time complexity using Big-$\mathcal{O}$ notation. You must mention the time complexity of *each* loop. — 04

```
int b = 0;
for (int i = n; i > 0; i -= 2) {
    for (int j = 1; j < n; j *= 2) {
        b += i * j;
    }
}
for (int k = 1; k <= n; k++) {
    for (int m = n; m > 1; m /= 3) {
        b += k - m;
    }
}
return b;
```

b. Consider the following functions. — 03

$$f_1(n) = \log\left(n^{2024}\right) \qquad f_4(n) = \sqrt{n}\,\log n$$
$$f_2(n) = n^2 \cdot \log_n(n^n) \qquad f_5(n) = n + n\log n$$
$$f_3(n) = 2^{2n} \qquad f_6(n) = n \cdot \sqrt[4]{n^2}$$

Write a correct asymptotic upper bound for each, and **sort** the functions in ascending order of their growth rate.

c. **Prove** (with proper explanation / a formula): $\sqrt[4]{10n^4 + 9n^3 + 8n} = \Theta(n)$. — 02

2. a.

Consider the following code snippet.

```
Function F(n):
    if (n <= 1):
        return 1;
    else:
        a = F(n/2);
        b = F(n/2);
        c = F(n/2);
        return a + b + c + G(n);

Function G(n):
    sum = 0;
    for (i = 1; i <= n; i++):
        sum = sum + i;
    return sum;
```

(i) **Write** the recurrence relation for the running time of function `F`. 02

(ii) **Solve** the recurrence from (i) and express the time complexity with an asymptotic (Big-$\mathcal{O}$) bound using a suitable method. 03

b. **Solve** the following recurrence relation. Give a closed form and then the asymptotic bound. 03

$$T(n) = 2\,T(n-1) + 1, \qquad T(0) = 1.$$

c. **Solve** the following recurrence relation. (The Master Theorem does not apply — use a recursion tree or the substitution method. Show your steps.) 04

$$T(n) = T(n/2) + T(n/3) + n.$$

3. a. Analyse the following code segment and **express** the time complexity using Big-$\mathcal{O}$ notation. State the complexity of *each* loop. 04

```
int count = 0;
for (int i = 1; i <= n; i++) {
    for (int j = i; j <= n; j++) {
        count++;
    }
}
for (int k = 1; k <= n; k = k * 3) {
    count++;
}
return count;
```

b. Using the **substitution method**, **prove** that the recurrence below is $\mathcal{O}(n \log n)$. Clearly state your guess, the inductive step, and the base case. 03

$$T(n) = 2\,T(n/2) + n, \qquad T(1) = 1.$$

c. Why do we ignore constant factors and lower-order terms when expressing the time complexity of an algorithm? **Briefly** explain in your own words. 01

— End of Questions —

CSE221
Assignment 2

Consider a farm with **n** cows of two colors, say, white cows and black cows. The cows are arranged in their barn in such a way that all white cows are kept first, followed by all black cows. Each of the cows is assigned a serial number. **You are to find the serial number of the first black cow.**

a. **Describe** how to convert the given information into an array of integers.
CO1

b. Given the array of integers from Q(2a), **present** a method with a time complexity **less than O(n)** to solve the mentioned problem in the paragraph. Use pseudocode/ executable code/ step-by-step logical instructions to show your method.
CO1

c. Suppose we implement Quicksort using a strategy where we choose the **pivot as the median of the first, last, and middle elements of a subarray**. To be specific, the pivot is selected as follows:
CO1

$$\text{pivot} = \text{median}(A[\text{st}], A[\text{end}], A[\text{mid}]) \quad \text{where} \quad \text{mid} = \left\lfloor \frac{\text{st} + \text{end}}{2} \right\rfloor$$

For example, consider the array A = [6, 2, 9, 1, 7, 3, 8].
In the initial Quicksort call, start = 0 gives A[0] = 6, end = 6 gives A[6] = 8, and mid index $= \frac{(0+6)}{2} = 3$ gives A[3] = 1. Among these three values—1, 6, and 8—the sorted order is (1, 6, 8), so the median, and thus the chosen pivot, is 6.

What is the worst-case input for this pivot selection strategy in Quicksort? **Generate** such a worst-case scenario with a 7-element array containing the integers 1 through 7, each used exactly once. Briefly explain why it leads to worst-case behavior.

Now consider a factory where **n** humanoid robots work. You are to sort the robots based on some criteria. Answer Q2(d) – (e) based on this.

1.

d. **CO1** Each robot can form connections with some of the other robots. That means each robot can form at most (n-1) connections. Which algorithm will be the fastest to sort the robots based on the number of connections they formed? **Write** the name and time complexity of the algorithm.

e. **CO1** Suppose, in the factory, there exists a system that can find the tallest robot in O(1) time. That means you are given a function, $find_tallest(A)$, which returns the tallest robot from a list called **A**, in constant time.

Considering this information, design an algorithm to sort the robots in descending order of their heights. **The time complexity of your algorithm must not exceed O(n).**

i. **Present** your algorithm using pseudocode/ executable code/ step-by-step logical instructions.
ii. **Write** the time complexity of your algorithm.

2.

In Rivendell, **n** voters are casting their votes for various political parties. The votes are recorded in an array, where each number represents a party's ID.

However, Rivendell follows a special rule: a party can only form the government if it receives more than $\lfloor \frac{n}{2} \rfloor$, that is, a strict majority. Otherwise, the election results in a deadlock, and a re-election must be held. For example, in the array [1,2,2,2,3,2,1], party 2 wins the majority since it got 4 votes (more than $\lfloor \frac{7}{2} \rfloor = 3$).

a. **CO1** Can two different parties each secure a majority in any given vote array? **Explain** why or why not.

b. **CO1** Given a vote array and a party ID, we need to determine whether the party has won the majority in the array. **Propose** an O(N) algorithm to verify this, where N is the number of voters.

c. **CO1** Now, we want to apply a divide and conquer approach to find the majority from a given vote array. Like merge sort, if we divide the vote array into two halves, can the majority element in each half be different from the majority of the entire array? **Justify** your answer with an example.

d. **CO1** Reuse your algorithm of Q3(b) to complete the following algorithm to find the majority party from the vote array. The algorithm should return -1 if there's no majority party. You can use either pseudo-code or step-by-step instructions.

```
function majority(arr):
      if length(arr) == 1:
            return arr[0]

      mid = length(arr) // 2
      left_majority = majority(arr[0 ... mid])
      right_majority = majority(arr[mid+1 ... length(arr)-1])

      # Merge Step: Reuse your algorithm of Q3(b) to complete
```

You are working with a weather monitoring station that records the temperature every hour for 12 hours during the night shift. During the night, the temperature gradually drops to reach the coldest hour, and then begins to rise again. Your task is to find the minimum temperature of the night, which corresponds to the coldest hour.

The station tracks temperature every hour over a 12-hour period. A sample pattern of the temperature reading per hour is given here:

Hour	1	2	3	4	5	6	7	8	9	10	11	12
Temperature (°C)	28	24	20	16	12	10	9	11	15	19	23	27

In this example, the temperature decreases from hour 1 to hour 7 (coldest at 9°C). Then it increases from hour 8 to hour 12.

a. **CO1** **Propose** a linear search approach to find the minimum temperature. Present the solution with pseudocode/programmable code/step-by-step logical instructions.

b. **CO3** Now **propose** a modification to binary search to solve the same problem. Present the solution with pseudocode/programmable code/step-by-step logical instructions.

3.

a **CO1** You are working on a grading system for a national standardized exam where students are marked out of 100. You need to sort millions of student scores as efficiently as possible. Speed is critical, and memory is not a major constraint. Comparisons are costly.

Which algorithm is the most suitable for the above scenario? **Explain** your choice.
Write the time complexity of your preferred algorithm.

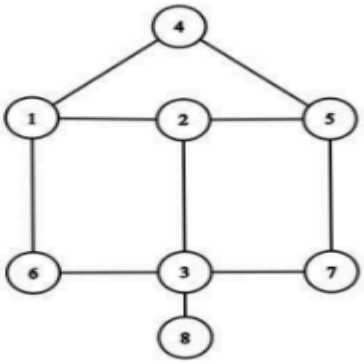
b **CO3** Suppose your friend wants to implement a variant of Merge Sort called Ternary Merge Sort, where instead of dividing the array into two halves, **he divides it into three equal (or nearly equal) parts** and recursively sort each part, and then merge the three sorted subarrays into a single sorted array. It is guaranteed that the input array will have a size of at least 3.

Now, he asks your help to **implement the 'dividing step' of the algorithm**. Present your idea with pseudocode/programmable-code/step-by-step logical instructions.

c **CO2** Does dividing into three subarrays make the algorithm asymptotically faster? **Explain** why or why not.

4.

Suppose, you and your classmates are trapped inside an ancient castle during a field trip. The only way out is to solve some tasks. The tasks involve navigating the castle and resolving ancient conflicts between the spirits that guard it. Your goal is to escape through the Gate of Reason before the castle resets and traps you forever.
There are 8 rooms in the castle and you can navigate from one room to another using some corridors. The map of the castle is given below:



5.

a **CO1** Suppose, you are in Room 4. Your first task is reaching Room 6 which is the control room and the Gate of Reason is also situated here. However, the castle is flooding and some of the corridors have been blocked. How can you determine the minimum number of steps required to reach Room 6 from Room 4 avoiding the blocked corridors?

Explain your idea in brief, *no need to write full code*.

b **CO1** Now that you have reached Room 6 and stopped the flood, all corridors are accessible once again. Only one challenge remains: unlocking the Gate of Reason to escape the castle. Each of the 8 rooms in the castle is guarded by a magical spirit. The ancient Gate of Reason only opens if the spirits can be divided into two opposing groups such that:

- No two spirits in the same group are in conflict (the rooms they are guarding should not be connected by corridors).
- The two groups must have the same number of spirits.

Propose a suitable algorithm to determine whether the Gate of Reason can be opened. Present your idea with pseudocode/programmable-code/step-by-step logical instructions.

You are participating in a competitive multiplayer strategy game where the objective is to defeat your opponent within a fixed time constraint. Throughout the game, various perks are provided at each level in the form of health boxes. At **Level 1**, you are presented with a list of health boxes, each represented by an integer. Positive values indicate a health boost to your own character, while negative values correspond to a reduction in your opponent's health. At this level, you are allowed to select exactly one box, and you want to maximize the health damage to your opponent. This list of health points is given below:

```
[2, 3, -4, -3, 5, -1, 2, -2]
```

Here, each element corresponds to a distinct health box.

6.

a **CO1** For the above scenario, can you find the box that will cause the maximum possible reduction in enemy health by applying a **divide and conquer** approach? **Explain** your idea in brief, *no need to write full code*.

b **CO3** At **Level 2**, you are offered the same collection of health boxes. However, this time you are allowed to choose **one or a block of consecutive boxes without gaps**. Now you want to increase your own health by collecting points that yield the highest points gain. You came up with an algorithm with **O(N*logN)** time complexity to compute this optimal health gain. Where 'N' is the number of health boxes.

Show a simulation of the algorithm with proper steps.

CSE221: Algorithms

Assignment 3 and 4

Total Marks: 100

Question 1 **(10 marks)**

A software release has eight tasks. An edge X -> Y means that task X must be completed before task Y.

Vertices: {A, B, C, D, E, F, G, H}

Edges: A->C, A->D, B->D, B->E, C->F, D->F, D->G, E->G, F->H, G->H

(a) Define a directed acyclic graph (DAG) and a topological ordering. State the condition under which a directed graph has a topological ordering. **[2 marks]**

(b) Use Kahn's algorithm to produce one topological ordering. Show the initial indegree of every vertex and the zero-indegree set after each removal. Break ties alphabetically.**[5 marks]**

(c) Suppose the dependency H -> D is added. Determine whether a topological ordering is still possible. Identify a resulting cycle and explain how Kahn's algorithm or DFS would detect the problem. **[3 marks]**

Question 2 **(10 marks)**

A directed communication network is represented by the following edge list. During every DFS, break ties alphabetically.

Vertices: {A, B, C, D, E, F, G, H}

Edges: A->B, B->C, C->A, C->D, D->E, E->D, E->F, F->G, G->F, G->H

(d) Starting at A, perform the first DFS of Kosaraju's algorithm. Record each vertex's discovery and finish time, then list vertices in decreasing finish-time order. **[3 marks]**

(e)Construct the transpose graph and perform the second DFS in that order. Show the DFS trees and report every strongly connected component (SCC). **[4 marks]**

(f) Contract every SCC into one super-node, draw or list the edges of the condensation graph, and give one topological ordering of this graph. Explain why it must be a DAG. **[2 marks]**

(g) State the time complexity of Kosaraju's algorithm with an adjacency-list representation. **[1 marks]**

Question 3 **(10 marks)**

A delivery service uses the directed network below. All weights are non-negative travel times.

Edge	Weight	Edge	Weight
S->A	4	S->B	2
B->A	1	A->C	3
B->C	5	B->D	8
C->D	2	C->T	7
D->T	1	A->T	12

(h) Distinguish the single-source shortest-path (SSSP) problem from the all-pairs shortest-path (APSP) problem. Identify which problem is being solved when S is the only source. **[2 marks]**

(i) Simulate Dijkstra's algorithm from S using a min-priority queue. After every extract-min, show all distance and predecessor updates. **[5 marks]**

(j) Write the final shortest path from S to T, its total cost, and the running time of Dijkstra's algorithm with an adjacency list and binary heap. **[2 marks]**

(k) Explain how repeated Dijkstra runs can solve APSP on this graph, and state the resulting asymptotic time complexity. **[1 marks]**

Question 4 **(10 marks)**

Place N queens on an N x N chessboard so that no two queens attack each other. For this question, use N = 8 and place one queen in each column.

(l) State the three conflict conditions that must be checked before placing a queen at row r and column c. Explain how rows and the two diagonal directions can be represented for fast safety checks. **[2 marks]**

(m) Write recursive backtracking pseudocode that tries rows 1 through 8 in each column, places a safe queen, recurses to the next column, and removes the queen when that branch fails. **[4 marks]**

(n) Give one valid arrangement of eight queens as a list where the value at position c is the selected row for column c. Verify briefly that no two listed queens share a row or diagonal. **[2 marks]**

(o) Explain how pruning improves on generating every board arrangement. State a reasonable worst-case time bound and the auxiliary-space complexity of the backtracking solution. **[2 marks]**

Question 5 (10 marks)

The following undirected graph represents possible fiber links and their installation costs. For equal weights, process edges in lexicographic order.

Edge	Weight	Edge	Weight
A-B	4	A-C	3
B-C	2	B-D	5
C-D	3	C-E	6
D-E	1	D-F	7
E-F	4	E-G	5
F-G	2		

(p) Define a minimum spanning tree (MST), and state two conditions the input graph must satisfy for a single spanning tree to exist. **[2 marks]**

(q) Simulate Kruskal's algorithm. Show the sorted edge order, whether each considered edge is selected or rejected, and the DSU components after each selected edge. Use union by rank and path compression. **[5 marks]**

(r) List the final MST edges, calculate the total cost, and verify the required number of edges. **[1 marks]**

(s) Suppose edge C-E must be included even if it raises the cost. Describe how to modify Kruskal's algorithm, then find the minimum-cost spanning tree under this constraint. **[2 marks]**

Question 6 (10 marks)

A lab has one machine and receives the following half-open reservation requests [start, finish).

A	B	C	D	E	F	G	H	I	J	K
[1,4)	[3,5)	[0,6)	[5,7)	[3,9)	[5,9)	[6,10)	[8,11)	[8,12)	[2,14)	[12,16)

(t) Apply the earliest-finish-time greedy algorithm. Show the sorted order, each accept/reject decision, and the maximum compatible set. **[4 marks]**

(u) Give a brief exchange-argument justification for why choosing the compatible interval that finishes earliest is optimal for the unweighted problem. **[2 marks]**

(v) For the weighted requests P=[1,3), value 5; Q=[3,5), value 5; and R=[1,5), value 13, show that earliest-finish-time greedy is not optimal. Explain why weighted interval scheduling needs a different method, and write the dynamic-programming recurrence using p(j), the last request compatible with j. **[4 marks]**

Question 7 **(10 marks)**

A telemetry system observes the following symbol frequencies. When frequencies tie, place the alphabetically smaller symbol or subtree on the left. Treat left as 0 and right as 1.

A	**B**	**C**	**D**	**E**	**F**
32	25	20	12	7	4

(w) Simulate Huffman's greedy algorithm using a min-priority queue. Show every merge and draw the final Huffman tree. **[4 marks]**

(x) Write the codeword for each symbol and calculate the total number of bits needed to encode all 100 symbol occurrences. **[3 marks]**

(y) Encode the message ABACA. Compare its length with fixed-length coding for six symbols and state the number of bits saved or lost. Ignore the code-table overhead. **[2 marks]**

(z)Explain briefly how the greedy choice in Huffman coding leads to an optimal prefix-free code. **[1 marks]**

Question 8 **(10 marks)**

In the 15-puzzle, the blank may swap with an adjacent tile. Consider the following initial and goal states.

	Initial state					**Goal state**		
1	2	3	4		1	2	3	4
5	6	7	8		5	6	7	8
9	10	11	12		9	10	11	12
_	13	14	15		13	14	15	_

(aa) Describe a state representation, the legal moves, and the inversion-parity test used to reject an unsolvable 4x4 puzzle before searching. **[2 marks]**

(bb) Write recursive backtracking pseudocode that uses a visited-state set, a path list, a depth limit, pruning, and undoing of moves. The procedure should return a solution path if one is found. **[4 marks]**

(cc) Using move order Right, Up, Left, Down, trace the successful branch from the initial state through the first three recursive levels and write the move sequence that reaches the goal. **[2 marks]**

(dd) Explain why plain depth-first backtracking does not guarantee the shortest solution. Name a method that does guarantee the shortest number of moves and compare its memory use with DFS backtracking. **[2 marks]**

Question 9 (10 marks)

A rescue aircraft has capacity 10 weight units. The available supply bundles are listed below.

Bundle	Weight	Value	Value/weight
A	2	60	30
B	4	100	25
C	6	120	20
D	5	95	19

(ee) Explain the difference between fractional and 0-1 knapsack, including whether an item may be divided and the standard algorithmic approach for each. **[2 marks]**

(ff) Solve the fractional-knapsack version using the greedy value-to-weight ratio. Show the selected fractions and maximum value. **[3 marks]**

(gg) For the 0-1 version, write the recurrence and base cases, construct the DP table for capacities 0 through 10, and report the selected bundles and maximum value. **[4 marks]**

(hh) Use your two results to explain why the fractional greedy rule is not guaranteed to solve 0-1 knapsack. **[1 marks]**

Question 10 (10 marks)

Let X = ABCBDAB and Y = BDCABA. For LCS(i, j), use the first i characters of X and the first j characters of Y.

(ii) Write the complete LCS recurrence relation, including base cases, and explain what each DP cell represents. **[3 marks]**

(jj) Construct the full dynamic-programming table, determine the LCS length, and backtrack to produce one valid LCS string. If several answers exist, any one is acceptable. **[4 marks]**

(kk) Explain overlapping subproblems and optimal substructure in the context of LCS. Refer to specific recursive subproblems to show how both properties arise. **[2 marks]**

(ll) State the DP time and space complexity. If only the LCS length is needed, state how the space can be reduced. **[1 marks]**